



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第八章 图



- 图的基本概念
- 图的存储结构
- 图的遍历与连通性
- 最小生成树
- 最短路径
- 活动网络



8.3 图的遍历与连通性



- 从已给的连通图中某一顶点出发，沿着一些边访问遍图中所有的顶点，且使每个顶点仅被访问一次，就叫做**图的遍历** (Graph Traversal)。
- 图中可能存在回路，且图的任一顶点都可能与其它顶点相通，在访问完某个顶点之后可能会沿着某些边又回到了曾经访问过的顶点。



- 为了避免重复访问，可设置一个标志顶点是否被访问过的辅助数组 **visited []**。
- 辅助数组visited[]的初始状态为 0, 在图的遍历过程中，一旦某一个顶点 i 被访问，就立即让visited[i]为 1, 防止它被多次访问。



- **图的遍历分类**

- ◆ **深度优先搜索**

DFS (Depth First Search)

- ◆ **广度优先搜索**

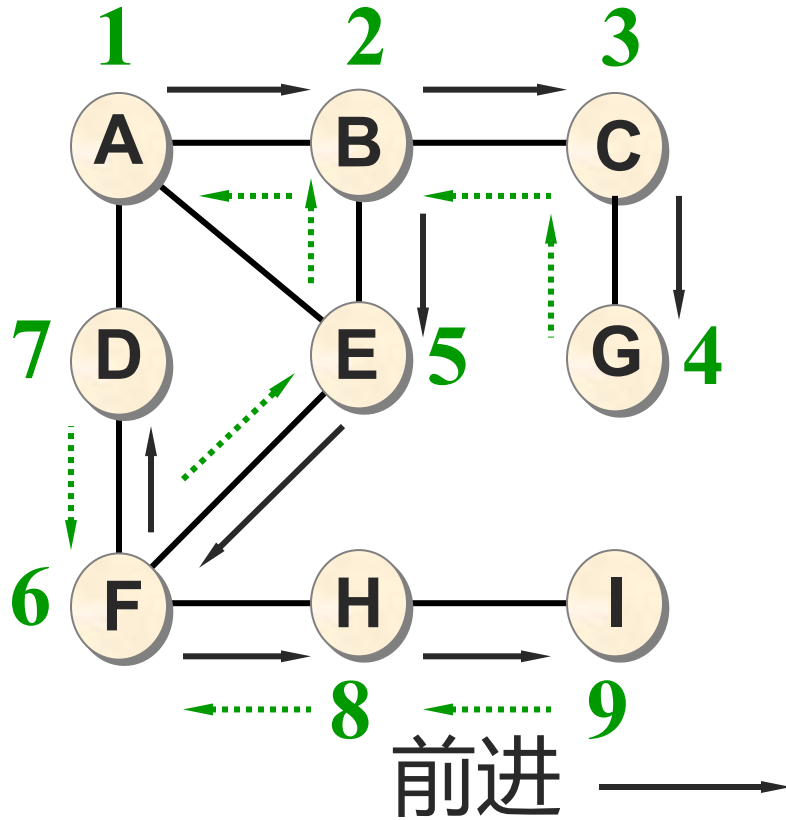
BFS (Breadth First Search)



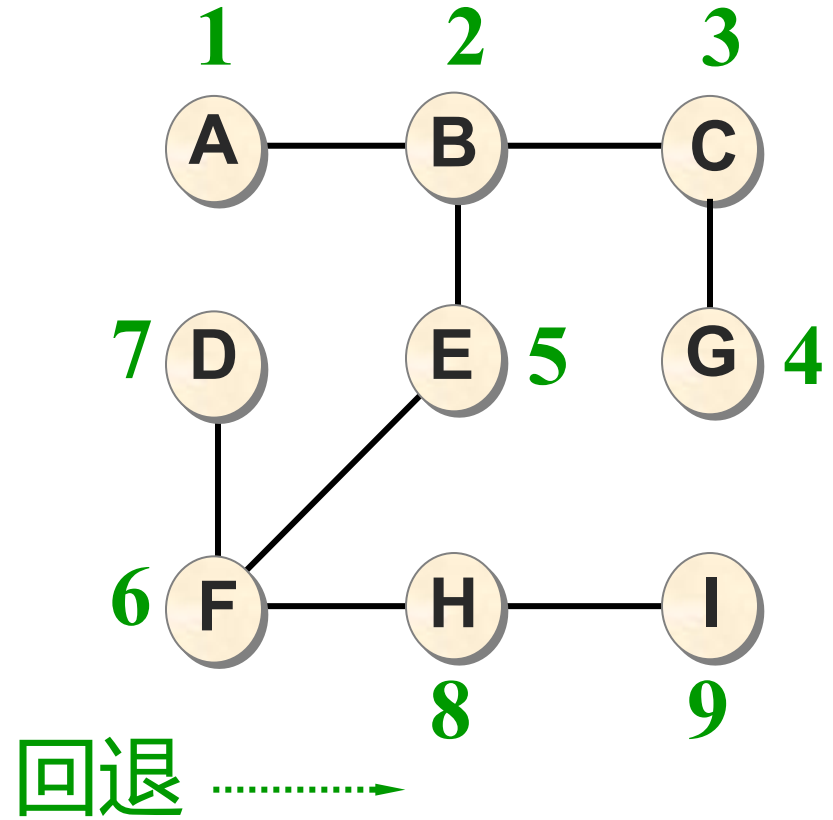
深度优先搜索DFS (Depth First Search)



深度优先搜索的示例



深度优先搜索过程



深度优先生成树



- **DFS** 在访问图中某一起始顶点 v 后,
 - 由 v 出发, 访问它的任一邻接顶点 w_1 ;
 - 再从 w_1 出发, 访问与 w_1 邻接但还没有访问过的顶点 w_2 ;
 - 然后再从 w_2 出发, 进行类似的访问, ... 如此进行下去, 直至到达所有的邻接顶点都被访问过的顶点 u 为止。
 - 接着, 退回一步, 退到前一次刚访问过的顶点, 看是否还有其它没有被访问的邻接顶点。如果有, 则访问此顶点, 之后再从此顶点出发, 进行与前述类似的访问; 如果没有, 就再退回一步进行搜索。
 - 重复上述过程, 直到连通图中所有顶点都被访问过为止。



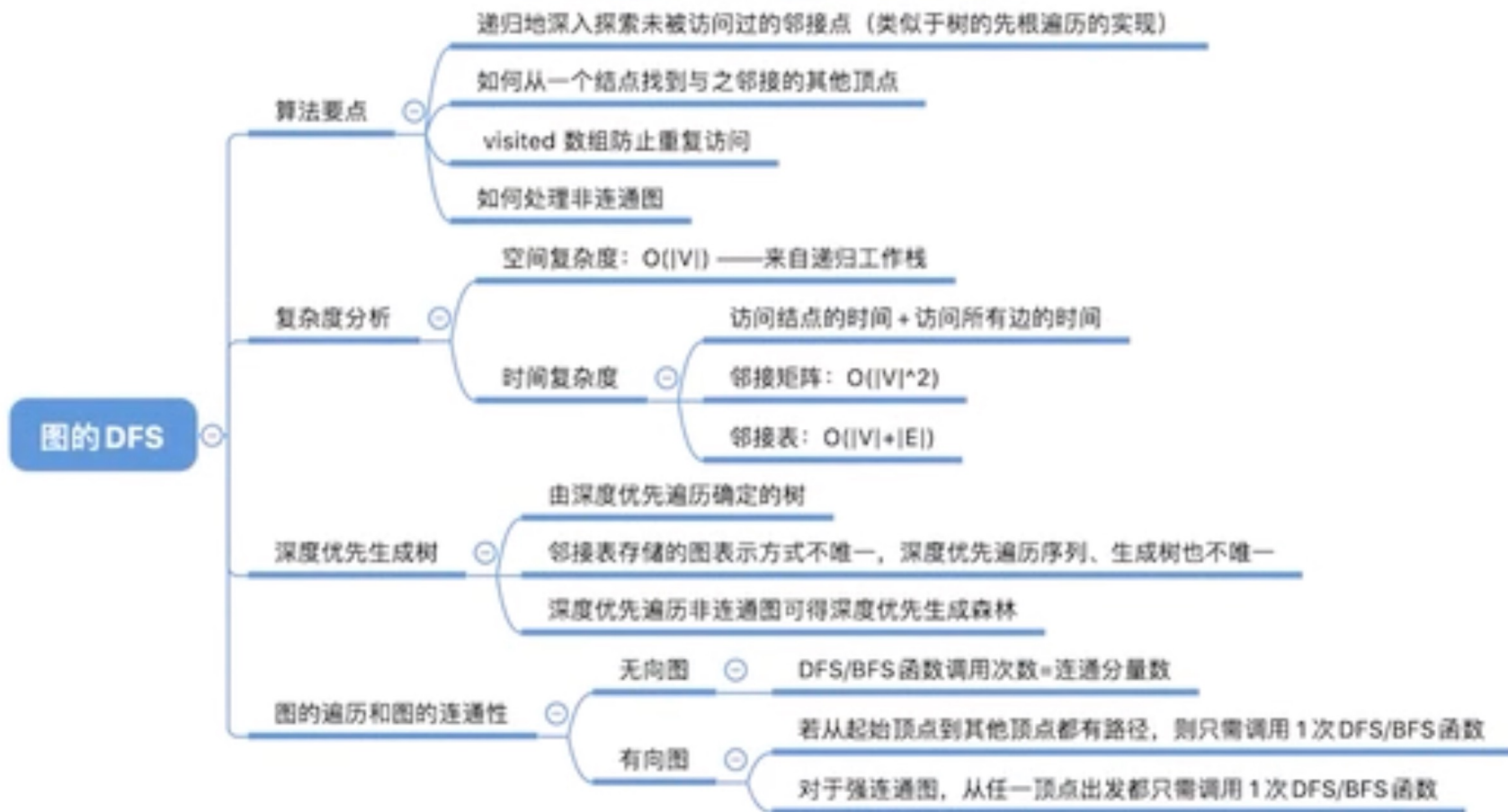
图的深度优先搜索算法



```
template<class T, class E>
void DFS (Graph<T, E>& G, const T& v) {
    //从顶点v出发对图G进行深度优先搜索的主过程
    int i, loc, n = G.NumberOfVertices();           //顶点个数
    bool *visited = new bool[n];                   //创建辅助数组
    for (i = 0; i < n; i++) visited [i] = false;
                                                //辅助数组visited初始化
    loc = G.getVertexPos(v);
    DFS (G, loc, visited); //从顶点loc开始深度优先搜索
    delete [] visited;                             //释放visited
};
```




```
template<class T, class E>
void DFS (Graph<T, E>& G, int v, bool visited[]) {
    cout << G.getValue(v) << ' ';    //访问顶点v
    visited[v] = true;                //作访问标记
    int w = G.getFirstNeighbor (v);    //第一个邻接顶点
    while (w != -1) {                  //若邻接顶点w存在
        if ( !visited[w] ) DFS(G, w, visited);
                                   //若w未访问过, 递归访问顶点w
        w = G.getNextNeighbor (v, w); //下一个邻接顶点
    }
};
```



深度优先遍历的序列是否唯一？

不是唯一的。给定某顶点，先往哪个方向探索都可以，即邻接顶点的顺序可以任意打乱，都是深度优先遍历。

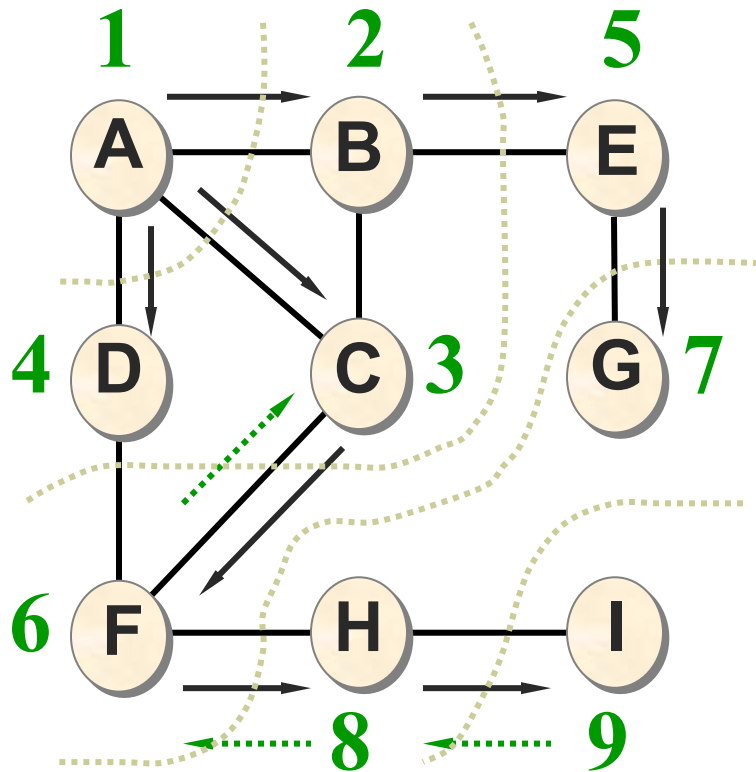
以树的遍历为例，“根 → 左 → 右” “左 → 根 → 右” “左 → 右 → 根” 分别对应前序、中序、后序遍历，它们展示了三种遍历优先级，然而这三者都属于深度优先遍历。



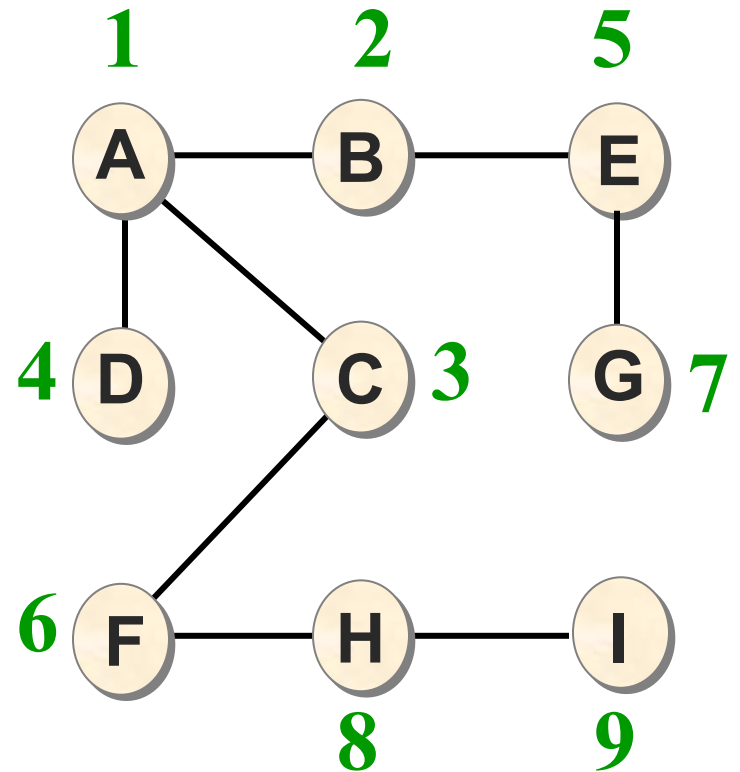
广度优先搜索BFS (Breadth First Search)



- 广度优先搜索的示例



广度优先搜索过程



广度优先生成树



- BFS在访问了起始顶点 v 之后,
 - 由 v 出发, 依次访问 v 的各个未被访问过的邻接顶点 $w_1, w_2, \dots, w_t$ 。
 - 然后再顺序访问 $w_1, w_2, \dots, w_t$ 的所有还未被访问过的邻接顶点。
 - 再从这些访问过的顶点出发, 再访问它们的所有还未被访问过的邻接顶点, ... 如此做下去, 直到图中所有顶点都被访问到为止。
- 广度优先搜索是一种**分层**的搜索过程, 每向前走一步可能访问一批顶点, 不像深度优先搜索那样有往回退的情况。
- 因此, **广度优先搜索不是一个递归的过程。**



- 为了实现逐层访问, 算法中使用了一个**队列**, 以记忆正在访问的这一层和上一层的顶点, 以便于向下一层访问。
- 为避免重复访问, 需要一个辅助数组 `visited [ ]`, 给被访问过的顶点加标记。

图的广度优先搜索算法

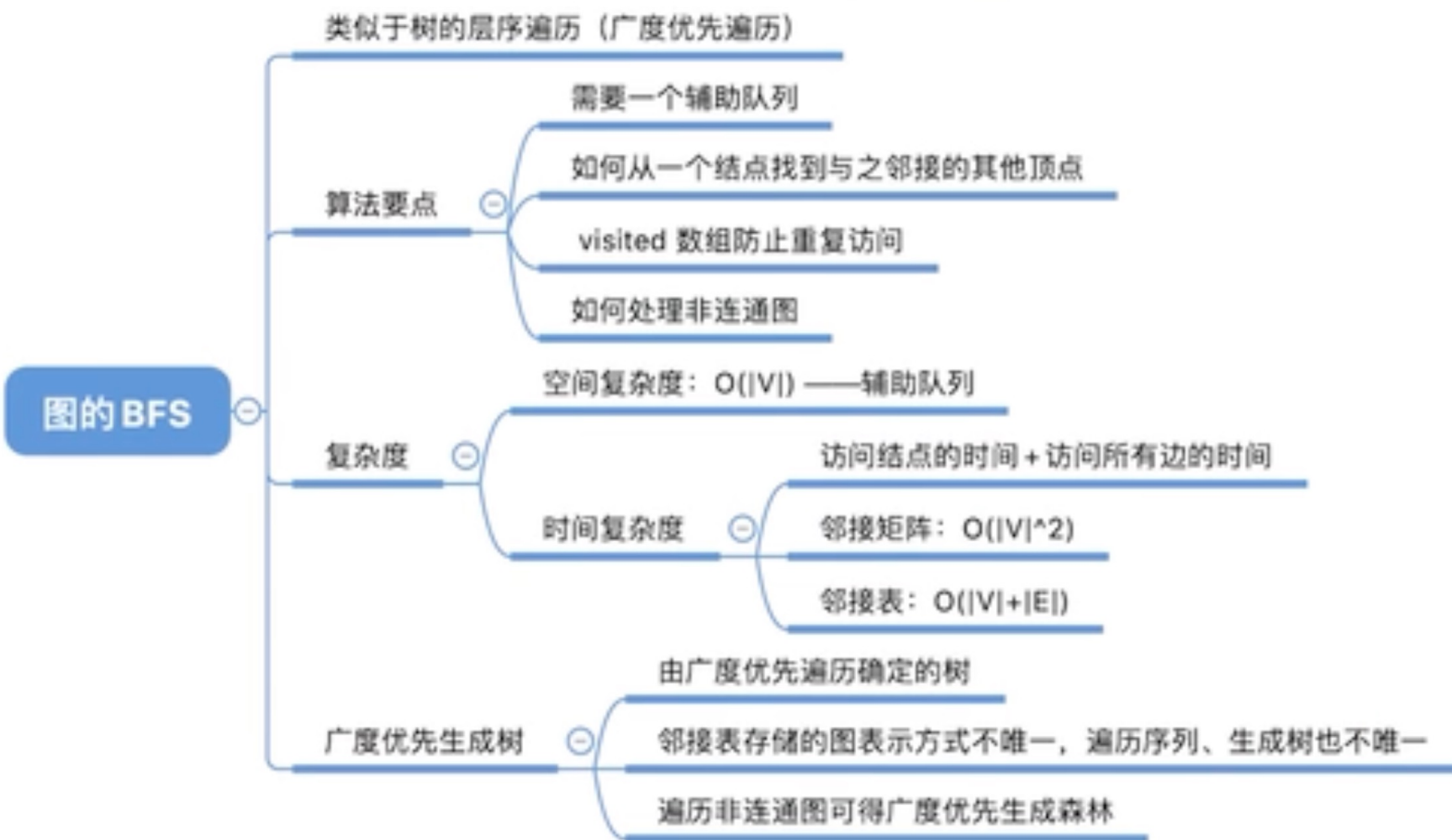
```
template <class T, class E>
void BFS (Graph<T, E>& G, const T& v) {
    int i, w, n = G.NumberOfVertices(); //图中顶点个数
```



```
bool *visited = new bool[n];  
for (i = 0; i < n; i++) visited[i] = false;  
int loc = G.getVertexPos (v);           //取顶点号  
cout << G.getValue (loc) << ' ';      //访问顶点v  
visited[loc] = true;                     //做已访问标记  
Queue<int> Q; Q.Enqueue (loc);           //顶点进队列, 实现分层访问  
while (!Q.IsEmpty() ) {                 //循环, 访问所有结点  
    Q.DeQueue (loc);  
    w = G.getFirstNeighbor (loc);        //第一个邻接顶点  
    while (w != -1) {                   //若邻接顶点w存在  
        if (!visited[w]) {              //若未访问过
```



```
    cout << G.getValue (w) << ' ';    //访问
    visited[w] = true;
    Q.Enqueue (w);                      //顶点w进队列
}
w = G.getNextNeighbor (loc, w);
                                //找顶点loc的下一个邻接顶点
}
//外层循环，判队列空否
delete [] visited;
};
```



广度优先遍历的序列是否唯一？

不唯一。广度优先遍历只要求按“由近及远”的顺序遍历，而多个相同距离的顶点的遍历顺序允许被任意打乱。



1. 深度优先遍历通常使用什么数据结构?

- A) 队列
- B) 栈
- C) 优先队列
- D) 数组

2. 广度优先遍历通常使用什么数据结构?

- A) 队列
- B) 栈
- C) 优先队列
- D) 数组

3. 对于同一个图，从同一个顶点出发，**DFS**和**BFS**的遍历序列：

- A) 一定相同
- B) 一定不同
- C) 可能相同，也可能不同
- D) 只有顶点数少时才相同

4. **DFS**遍历图(以邻接表存储)的时间复杂度是：

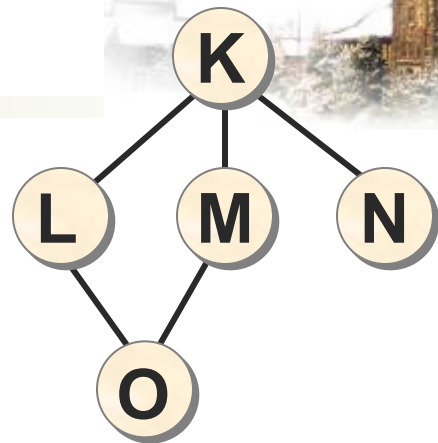
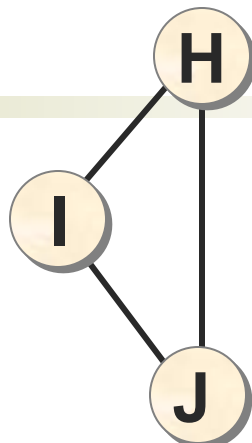
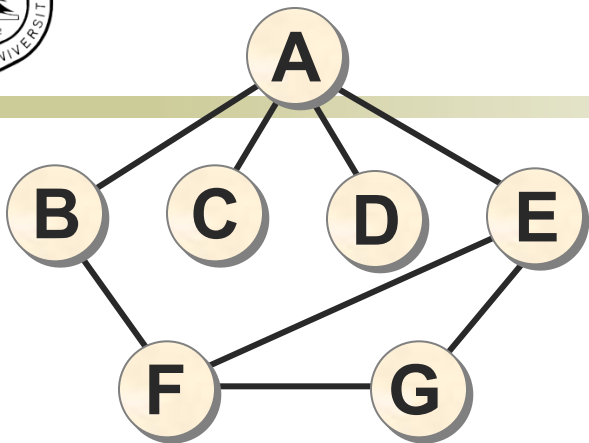
- A) $O(n)$
- B) $O(n^2)$
- C) $O(n + e)$
- D) $O(n \times e)$



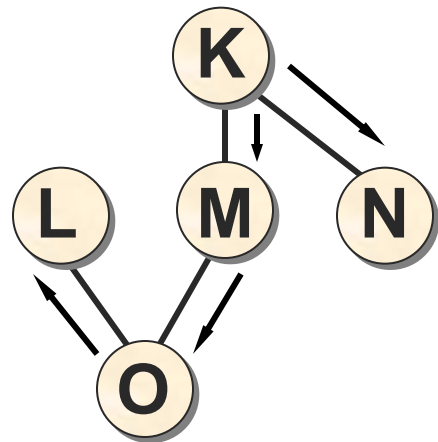
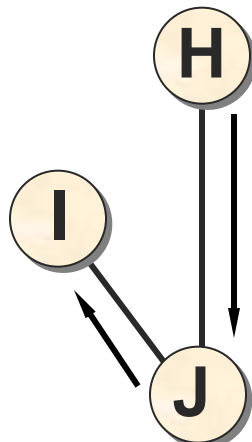
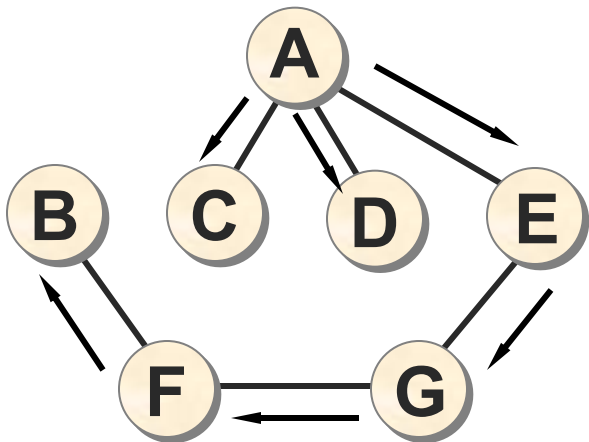
连通分量 (Connected component)



- 当无向图为**非连通图**时，从图中某一顶点出发，利用深度优先搜索算法或广度优先搜索算法不可能遍历到图中的所有顶点，只能访问到该顶点所在**极大连通子图**（连通分量）的所有顶点。
- 若从无向图每一连通分量中的一个顶点出发进行遍历，可求得无向图的所有连通分量。
- 例如，对于非连通的无向图，所有连通分量的生成树组成了非连通图的**生成森林**。



非连通无向图



非连通图的生成森林



确定连通分量的算法



```
template <class T, class E>
void Components (Graph<T, E>& G) {
    //通过DFS, 找出无向图的所有连通分量
    int i, n = G.NumberOfVertices();    //图中顶点个数
    bool *visited = new bool[n];        //访问标记数组
    for (i = 0; i < n; i++) visited[i] = false;
    for (i = 0; i < n; i++)              //扫描所有顶点
        if (!visited[i]) {               //若没有访问过
            DFS (G, i, visited);          //访问
            OutputNewComponent();          //输出连通分量
        }
    delete [ ] visited;
}
```